JAVA PROGRAMMING / DSA

Chapter 43: Binary Search in Depth

Personal Notes

Binary search on a sorted array is a classic: cut the search space in half each step, done in O(log n). That's the tip of the iceberg. The REAL power comes from a mindset shift — you can binary-search over the SEARCH SPACE OF ANSWERS, not just an array.

"Minimum time to eat all bananas." "Smallest capacity that lets us ship in D days." "Kth smallest in two sorted arrays." These don't look like search problems at first — but each has a MONOTONIC predicate hiding inside. Once you spot it, binary search collapses them from $O(n^2)$ or worse into O(n log MAX). This chapter covers the classical patterns AND the search-on-answer-space technique that unlocks a huge swath of interview problems.

1. Classical Binary Search Refresher

Given a sorted array and a target, find the target's index (or determine it's not present) in O(log n). Halve the search space each step.

```
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;    // avoid overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Two Critical Details

- Compute mid as left + (right - left) / 2, NOT (left + right) / 2 — the latter can overflow for large indices
- Use left ≤ right (inclusive) to handle the case where the target might be at either boundary

2. The Real Skill — Recognizing the MONOTONIC PROPERTY

Binary search works whenever the search space has a MONOTONIC PROPERTY: as you move in one direction, some yes/no property switches ONCE from "no" to "yes" (or vice versa) — never flipping back.

```
    Example — search space of eating speeds:

    speed:  1  2  3  4  5  6  7  8  9  10 11 12
    can finish in time?:  N  N  N  N  N  Y  Y  Y  Y  Y  Y  Y
```

```
The predicate "can finish in time" flips from N to Y once.
Binary search finds that boundary in O(log n) checks.

Answer: smallest Y, which is speed 6.
```

The mindset shift: You're not searching for a specific VALUE in an array — you're searching for the BOUNDARY between "no" and "yes" answers to a question. That question is a helper function, often called canDo(x). If canDo(x) has monotonic behavior in x, binary search applies.

3. The Universal Template

Almost every binary search problem — array-based OR answer-space — fits this shape. Once you internalize it, most problems become fill-in-the-blank.

```
int binarySearch(int lo, int hi) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canDo(mid)) hi = mid;      // this mid works — try smaller
        else            lo = mid + 1;   // this mid fails — need bigger
    }
    return lo;
}
```

Three key decisions when adapting this template:

1. Define the SEARCH RANGE — [lo, hi] representing all possible answers
2. Define canDo(x) — a boolean function with monotonic behavior in x
3. Decide DIRECTION — do you want the smallest x with canDo(x)=true (as above), or the largest x with canDo(x)=false? Adjust accordingly.

Why lo < hi (strict) and hi = mid (not mid - 1): This template searches for the smallest x where canDo(x) is true. Using strict inequality and hi = mid guarantees we never overshoot — the loop terminates with lo == hi at the boundary. Great for "minimum/smallest satisfying" problems. For "find exact target" problems, use the classical <= template with mid+1 / mid-1.

4. The Two Common Templates

Two shapes cover almost everything you'll see. Learn to pick the right one for the problem.

4.1 Template A — Exact Match (Classical)

```
int lo = 0, hi = n - 1;
while (lo <= hi) {                    // ≤ inclusive
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
```

```
    else hi = mid - 1;
}
return -1;
```

Use when: you want a specific value in a sorted array. Returns -1 if not present.

4.2 Template B — Find Boundary (Search on Answer Space)

```
int lo = MIN_POSSIBLE, hi = MAX_POSSIBLE;
while (lo < hi) {                       // < strict
    int mid = lo + (hi - lo) / 2;
    if (canDo(mid)) hi = mid;           // mid works, try smaller
    else lo = mid + 1;                   // mid fails, need bigger
}
return lo;
```

Use when: you want the smallest (or largest) value satisfying some property. Returns the boundary — the first (or last) x where canDo(x) flips.

> **The templates are NOT interchangeable:** Mixing the boundaries (using <= with hi = mid, for instance) causes infinite loops. Pick ONE template based on the problem shape and use it exactly. Don't invent hybrid boundary logic mid-solve.

5. Classical Problem 1 — First and Last Occurrence

Given a SORTED array (possibly with duplicates) and a target, find the FIRST and LAST indices where the target appears. Two boundary searches.

```
public int[] searchRange(int[] arr, int target) {
    int first = findBoundary(arr, target, true);
    int last  = findBoundary(arr, target, false);
    return new int[]{first, last};
}

private int findBoundary(int[] arr, int target, boolean findFirst) {
    int lo = 0, hi = arr.length - 1, result = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) {
            result = mid;
            if (findFirst) hi = mid - 1;   // keep looking left
            else           lo = mid + 1;    // keep looking right
        } else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return result;
}

// Time: O(log n)
```

Dry Run for arr = [5, 7, 7, 8, 8, 10], target = 8

```
     findFirst = true:
       lo=0, hi=5, mid=2, arr[2]=7 < 8 → lo=3
       lo=3, hi=5, mid=4, arr[4]=8 = target → result=4, hi=3 (keep going
left)
       lo=3, hi=3, mid=3, arr[3]=8 = target → result=3, hi=2 (keep going
left)
       lo=3 > hi=2, stop.  first = 3

     findFirst = false:
       lo=0, hi=5, mid=2, arr[2]=7 < 8 → lo=3
       lo=3, hi=5, mid=4, arr[4]=8 = target → result=4, lo=5 (keep going
right)
       lo=5, hi=5, mid=5, arr[5]=10 > 8 → hi=4
       lo=5 > hi=4, stop.  last = 4

     Answer: [3, 4]
```

6. Classical Problem 2 — Search in Rotated Sorted Array

A sorted array was rotated at some pivot. Find a target in O(log n). This is a Google interview favorite — one of the hardest classical binary search variants.

The Key Insight

After rotation, ONE half around the pivot is still sorted. Determine which half is sorted, then check if the target lies in it. That half is either the answer's location or safely discarded.

```
public int search(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;

        // Is left half sorted?
        if (arr[lo] <= arr[mid]) {
            if (arr[lo] <= target && target < arr[mid]) hi = mid - 1;
            else lo = mid + 1;
        }
        // Otherwise right half is sorted
        else {
            if (arr[mid] < target && target <= arr[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Why This Works

```
Original sorted:  [1, 2, 3, 4, 5, 6, 7]
Rotated at 3:     [4, 5, 6, 7, 1, 2, 3]

mid = 3, arr[mid] = 7
Is arr[lo=0]=4 ≤ arr[mid]=7? YES → left half [4,5,6,7] is sorted.

If target is in [4, 7), search left. Otherwise search right.

After one comparison, we know which half to eliminate — same as
classical binary search, but with an extra sortedness check.
```

Related: Find Minimum in Rotated Sorted Array: Similar approach — the minimum is on the side where arr[mid] > arr[hi]. Answer: hi. That single condition, applied repeatedly, converges to the rotation point in O(log n).

7. Search on Answer Space — The Big Pattern

This is the pattern that transforms binary search from "cool array trick" to "go-to interview tool." Instead of searching in an array, you search over the RANGE OF POSSIBLE ANSWERS.

The Recipe

4. Ask: "What am I trying to minimize or maximize?" That's your ANSWER
5. Define the MIN and MAX possible values of the answer (this is your search range)
6. Write canDo(x) — given a candidate answer x, can we achieve it? Must be MONOTONIC
7. Binary search over the range using canDo(x) as the predicate

The monotonicity requirement is everything: canDo(x) must be TRUE for all x above some threshold and FALSE below (or vice versa). If canDo can flip back and forth, binary search doesn't apply. Always sanity-check: "if x works, does x+1 also work?" If yes, monotonic → binary search.

8. Classic Problem 3 — Koko Eating Bananas

Koko has N piles of bananas and H hours to finish them all. Each hour she picks a pile and eats up to K bananas from it (if the pile has fewer, she eats what's there and moves on). Find the MINIMUM K that lets her finish in time. LeetCode 875 — the definitive search-on-answer-space problem.

Recognize the Pattern

- Answer = eating speed K
- Range = [1, max(piles)] — can't be 0, no need to go higher than the biggest pile
- canDo(K) = "can Koko finish all piles in ≤ H hours at speed K?"
- Monotonic — if she can finish at speed K, she can also finish at any speed > K

```
public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canFinish(piles, mid, h)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

private boolean canFinish(int[] piles, int speed, int h) {
    long hours = 0;
    for (int p : piles) {
        hours += (p + speed - 1) / speed;   // ceil division
    }
    return hours <= h;
}

// Time: O(n log(max)) — log(max) iterations × O(n) per canFinish check
```

The ceiling division trick: (p + speed - 1) / speed gives ⌈p / speed⌉ using only integer arithmetic. This is the standard way to compute "hours for one pile" without floats. Adding speed-1 before dividing rounds up.

9. Classic Problem 4 — Capacity to Ship Packages in D Days

Given package weights that must be shipped IN ORDER, find the minimum ship capacity that lets you finish in D days. Same pattern as Koko: binary search on capacity.

Setup

- Answer = ship capacity
- Range = [max(weights), sum(weights)] — must fit any single package; sum is the trivial upper bound (1 day)
- canDo(capacity) = "can we ship in ≤ D days with this capacity?"

```
public int shipWithinDays(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) {
        lo = Math.max(lo, w);   // must handle biggest package
        hi += w;                 // trivial upper: 1 day
    }

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canShip(weights, mid, days)) hi = mid;
```

```
        else lo = mid + 1;
    }
    return lo;
}

private boolean canShip(int[] weights, int cap, int days) {
    int used = 1, current = 0;
    for (int w : weights) {
        if (current + w > cap) {
            used++;
            current = 0;
        }
        current += w;
    }
    return used <= days;
}
```

10. Classic Problem 5 — Find the Kth Smallest in Two Sorted Arrays

Given two sorted arrays, find the K-th smallest element overall in O(log(m + n)). Or the harder LeetCode 4: MEDIAN of two sorted arrays.

The Idea

Binary search the partition. Split each array into a left and right half so that all left elements are ≤ all right elements. The kth smallest is then the max of the two left maxes.

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B, A);   //
ensure A ≤ B

    int m = A.length, n = B.length;
    int total = (m + n + 1) / 2;
    int lo = 0, hi = m;

    while (lo <= hi) {
        int i = (lo + hi) / 2;          // partition point in A
        int j = total - i;               // partition point in B

        int aLeft  = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
        int aRight = (i == m) ? Integer.MAX_VALUE : A[i];
        int bLeft  = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
        int bRight = (j == n) ? Integer.MAX_VALUE : B[j];

        if (aLeft <= bRight && bLeft <= aRight) {
            if ((m + n) % 2 == 1) return Math.max(aLeft, bLeft);
```

```
            return (Math.max(aLeft, bLeft) + Math.min(aRight, bRight)) /
2.0;
        } else if (aLeft > bRight) hi = i - 1;
        else lo = i + 1;
    }
    return 0.0;   // unreachable
}

// Time: O(log(min(m, n)))
```

This is a legendary hard problem. Even understanding the setup is worth it — you'll rarely need to code it from scratch, but the technique (binary search on the partition) generalizes to other advanced problems.

11. Classic Problem 6 — Median of Row-Wise Sorted Matrix

Given a matrix where every row is sorted, find the median. Binary search on the ANSWER VALUE (not indices). For each candidate median x, count elements ≤ x in the matrix — use each row's binary search.

```
public int findMedian(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length;
    int lo = Integer.MAX_VALUE, hi = Integer.MIN_VALUE;
    for (int[] row : matrix) {
        lo = Math.min(lo, row[0]);
        hi = Math.max(hi, row[cols - 1]);
    }

    int target = (rows * cols) / 2;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        int countLE = 0;
        for (int[] row : matrix) {
            countLE += countLessOrEqual(row, mid);
        }
        if (countLE <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

private int countLessOrEqual(int[] row, int target) {
    int lo = 0, hi = row.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (row[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

This is a POWERFUL pattern — binary search on the answer's VALUE, count-based canDo(). Applies to "find kth" problems in matrices, sorted lists of pairs, and more.

12. Classic Problem 7 — Split Array Largest Sum

Split an array into K contiguous subarrays such that the LARGEST SUBARRAY SUM is minimized. LeetCode 410 — direct application of search-on-answer-space.

```
public int splitArray(int[] nums, int k) {
    int lo = 0, hi = 0;
    for (int n : nums) {
        lo = Math.max(lo, n);
        hi += n;
    }

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canSplit(nums, mid, k)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

private boolean canSplit(int[] nums, int maxSum, int k) {
    int splits = 1, current = 0;
    for (int n : nums) {
        if (current + n > maxSum) {
            splits++;
            current = 0;
        }
        current += n;
    }
    return splits <= k;
}
```

Same shape as Capacity to Ship — greedy splitting inside canDo() combined with binary search on the answer. Once you see the pattern, related problems all feel familiar.

13. Common Binary Search Patterns

Problem shape	Approach
Find exact value in a sorted array	**Classical Template A**
Find first / last occurrence of a value	**Two Template A searches**
Search in rotated sorted array	**Check which half is sorted**

Problem shape	Approach
Find peak / valley in an unsorted array	**Compare arr[mid] with arr[mid+1]**
Find sqrt / cube root of an integer	**Binary search over [0, n]**
Minimize max value over splits	**Binary search on answer + greedy check**
Maximize min value over choices	**Binary search on answer + feasibility check**
Find kth smallest in sorted matrix	**Binary search on value + count check**
Median of two sorted arrays	**Partition-based binary search**
Painter / carpenter allocation	**Binary search on max time/paint**
Aggressive cows (max min distance)	**Binary search on distance + feasibility**

14. When Search-on-Answer-Space Applies

Signal words to watch for

- "Minimum … such that …" or "maximum … such that …"
- "Smallest capacity / speed / time that lets us do X"
- "Split into K parts, minimize the maximum …"
- "K-th smallest / largest" (matrix, two sorted arrays, etc.)
- The problem asks for an OPTIMAL VALUE, not a specific index or element

Requirements

- Answer has a clear numeric range [MIN, MAX]
- You can write a canDo(x) function in polynomial time
- canDo(x) is MONOTONIC — flips exactly once across the range

> **The signature move:** When you see "minimize the maximum" or "maximize the minimum," that's binary search on the answer space almost every time. This phrasing is the strongest signal in DSA that binary search applies to a non-array problem.

15. Complexity Analysis

Problem	Time	Space
Classical binary search	O(log n)	O(1)
First / last occurrence	O(log n)	O(1)

Problem	Time	Space
Rotated sorted array	O(log n)	O(1)
Peak / valley finding	O(log n)	O(1)
Search on answer space	O(n × log(MAX))	O(1)
Median of two sorted arrays	O(log(min(m, n)))	O(1)
Kth smallest in sorted matrix	O(n × log(MAX_VAL))	O(1)

Binary search on answer space is O(n × log(MAX)) — the outer log iterations, each with an O(n) canDo() check. For MAX values up to 10^9, that's about 30 iterations — extraordinarily fast.

16. Common Mistakes

Mistake 1: Overflow in mid calculation

```
int mid = (lo + hi) / 2;              // ✗ can overflow for large indices
int mid = lo + (hi - lo) / 2;         // ✓ safe
int mid = (lo + hi) >>> 1;             // ✓ unsigned shift also safe
```

Mistake 2: Infinite loop from wrong boundary update

```
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (canDo(mid)) hi = mid;
    else lo = mid;               // ✗ INFINITE LOOP if mid == lo
}

// Fix: lo = mid + 1  (always move past mid when it fails)
```

Mistake 3: Wrong template for the problem

Using <= with hi = mid gives infinite loops. Using < with hi = mid - 1 misses the answer. Pick a template and use it exactly. Don't mix boundary styles.

Mistake 4: Non-monotonic predicate

```
// Bad: canDo(x) that returns YES, NO, YES for increasing x
// Binary search won't find the right answer — it assumes monotonicity.

// Sanity check: pick two x values, one small, one large.
// Does canDo flip exactly once between them? If yes, monotonic.
```

Mistake 5: Wrong search-space bounds

For Ship in D Days, lo must be at least max(weights) — otherwise a single package won't fit and canDo will never succeed. For Koko, lo=0 means "eat nothing per hour" — always false. Start bounds carefully.

Mistake 6: Not verifying canDo() correctness

The elegance of search on answer space depends entirely on canDo() being right. Test canDo(x) manually on a small input BEFORE wrapping in binary search. If canDo is buggy, the search is silently wrong.

Mistake 7: Forgetting return value convention

```
// Template B returns 'lo' after loop ends — the smallest answer where
canDo is true.
// If NO value satisfies canDo, lo will be hi+1 or past the range. Handle
it:
return lo <= MAX_POSSIBLE ? lo : -1;
```

17. Best Practices

- Always use lo + (hi - lo) / 2 for mid — never (lo + hi) / 2
- Match your template to your problem type — exact vs boundary
- For "minimize/maximize" problems, ask "can I do it with value x?" — that's canDo
- Sanity-check monotonicity before writing the loop
- Initialize lo and hi to true minimum/maximum possible answers, not arbitrary values
- Test canDo() on small inputs by hand before wrapping in binary search
- For rotated arrays, always check which HALF is sorted before deciding direction
- For counting-based canDo (Kth smallest), use non-strict ≤ carefully

18. Quick Reference

Concept	Meaning
Classical binary search	Find exact value in a sorted array
Template A	<= inclusive, mid+1 / mid-1 boundaries
Template B	< strict, hi = mid boundary (for finding first-yes)
Safe mid	lo + (hi - lo) / 2 to avoid overflow
Monotonic predicate	canDo(x) flips once from N to Y (or Y to N)
Search on answer space	Binary search over the range of possible answers
Signal phrase	"minimize maximum" or "maximize minimum"

Concept	Meaning
Rotated array trick	One half is always sorted — check and eliminate
Ceil division	(a + b - 1) / b in integer arithmetic
Time complexity	O(log n) for arrays, O(n log MAX) for answer space

19. Practice Problems

Try in order — foundational first, then rotated variants, then search-on-answer-space classics.

Foundational

8. Binary Search (LeetCode 704) — exact match.
9. First Bad Version (LeetCode 278) — classic boundary search.
10. Find First and Last Position of Element in Sorted Array (LeetCode 34).
11. Search Insert Position (LeetCode 35).

Peak / Rotated

12. Find Peak Element (LeetCode 162).
13. Find Minimum in Rotated Sorted Array (LeetCode 153).
14. Find Minimum in Rotated Sorted Array II — with duplicates (LeetCode 154).
15. Search in Rotated Sorted Array (LeetCode 33).
16. Search in Rotated Sorted Array II (LeetCode 81).

Search on Answer Space (The Big Group)

17. Koko Eating Bananas (LeetCode 875).
18. Capacity To Ship Packages Within D Days (LeetCode 1011).
19. Split Array Largest Sum (LeetCode 410).
20. Minimum Number of Days to Make m Bouquets (LeetCode 1482).
21. Minimize Max Distance to Gas Station (LeetCode 774).
22. Aggressive Cows — max the minimum distance between placed cows.
23. Painter's Partition Problem — minimize max time.
24. Magnetic Force Between Two Balls (LeetCode 1552).

Advanced

25. Median of Two Sorted Arrays (LeetCode 4).
26. Kth Smallest Element in a Sorted Matrix (LeetCode 378).

27. Find K-th Smallest Pair Distance (LeetCode 719).
28. Search a 2D Matrix (LeetCode 74).
29. Search a 2D Matrix II (LeetCode 240).
30. Sqrt(x) (LeetCode 69).
31. Valid Perfect Square (LeetCode 367).
32. Find K Closest Elements (LeetCode 658).

20. Final Takeaway

Binary search is not one algorithm — it's a MINDSET. The moment you see a problem with a monotonic yes/no property, binary search collapses it from $O(n^2)$ or worse into O(n log MAX). The hardest part is recognizing that monotonicity is hidden in the problem — once you see it, the code is boilerplate.

For interviews, master five problems: standard binary search (Template A), first/last occurrence (boundary search), rotated sorted array (which half is sorted?), Koko Eating Bananas (search on answer space), and Median of Two Sorted Arrays (advanced partitioning). Those five hit every major variant and prepare you for the entire binary search category.

> **Key Takeaway:** Binary search halves the search space when a monotonic property exists. Use Template A (≤, mid±1) for exact match, Template B (<, hi = mid) for finding the FIRST value where a property flips. Search on ANSWER SPACE unlocks a huge class of "minimize the maximum" / "maximize the minimum" problems — define range, write canDo(), binary search. Signal phrase: "minimize max" or "maximize min" ⇒ almost always binary search on answer space.

21. What's Next?

With binary search patterns done, the remaining specialized DSA topics are:

- KMP / Rabin-Karp — string pattern matching in O(n + m)
- Segment Tree / Fenwick Tree — range queries WITH point updates
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points
- Number Theory basics — modular arithmetic, primes, GCD tricks
- Randomized Algorithms — quickselect, reservoir sampling
- Suffix Arrays and Automata — for advanced string problems
- System Design fundamentals — the natural big step for senior interviews